VINUNIVERSITY

Lecture 6: Java Static Classes and Members

Static Classes and Static Members

Hieu Pham, College of Engineering & Computer Science, VinUniversity

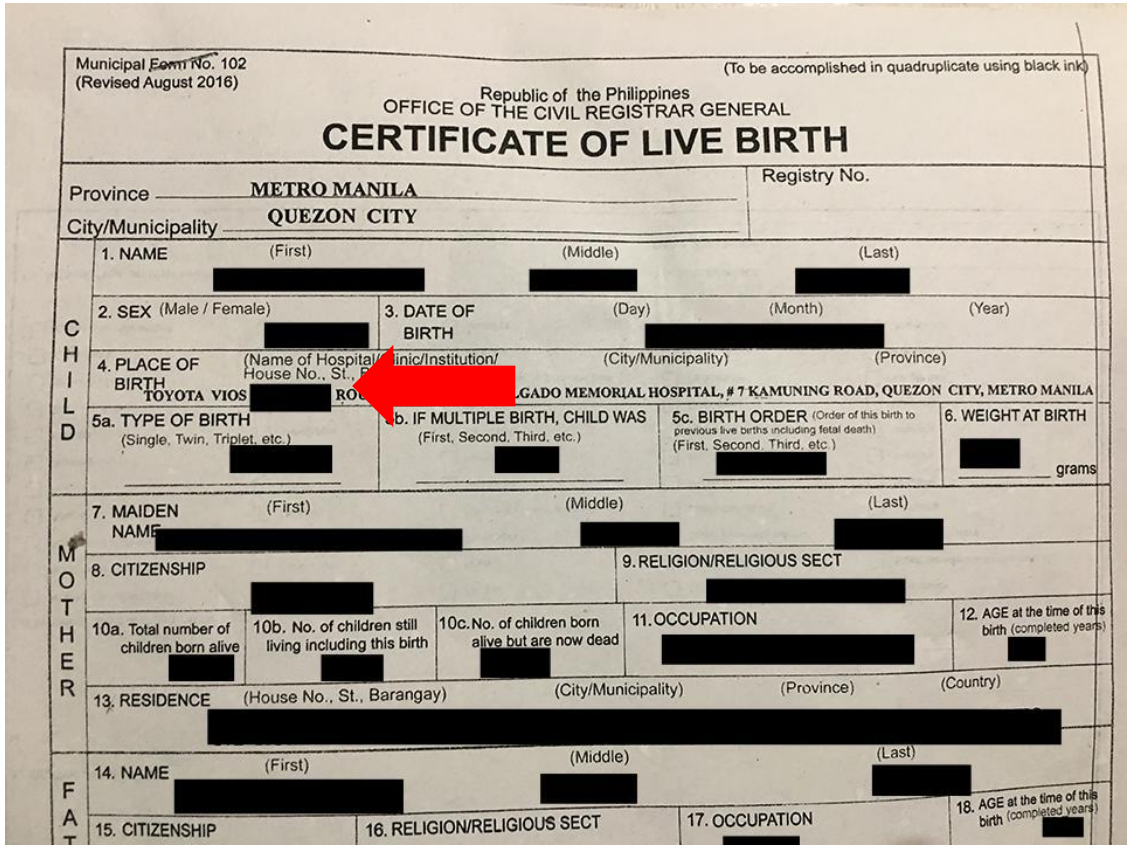
March 04, 2025

LECTURE 05 (REVISIT)

Lecture 05 aims to:

- understand and use constructors in Java.
- understand and implement inheritance types, such as single, multi-level, and hierarchical inheritance.
- understand how to use interface to implement hybrid and multiple inheritance in Java.

MOTIVATION EXAMPLE



Municipal Form No. 102
(Revised August 2016)

(To be accomplished in quadruplicate using black ink)

Republic of the Philippines
OFFICE OF THE CIVIL REGISTRAR GENERAL

CERTIFICATE OF LIVE BIRTH

Province: METRO MANILA
City/Municipality: QUEZON CITY
Registry No.

CHILD

1. NAME (First) (Middle) (Last)
2. SEX (Male / Female)
3. DATE OF BIRTH (Day) (Month) (Year)
4. PLACE OF BIRTH (Name of Hospital/Clinic/Institution/House No., St., ...) (City/Municipality) (Province)
TOYOTA VIOS ... GADO MEMORIAL HOSPITAL, # 7 KAMUNING ROAD, QUEZON CITY, METRO MANILA
5a. TYPE OF BIRTH (Single, Twin, Triplet, etc.)
5b. IF MULTIPLE BIRTH, CHILD WAS (First, Second, Third, etc.)
5c. BIRTH ORDER (Order of this birth to previous live births including fetal death) (First, Second, Third, etc.)
6. WEIGHT AT BIRTH grams

MOTHER

7. MAIDEN NAME (First) (Middle) (Last)
8. CITIZENSHIP
9. RELIGION/RELIGIOUS SECT
10a. Total number of children born alive
10b. No. of children still living including this birth
10c. No. of children born alive but are now dead
11. OCCUPATION
12. AGE at the time of this birth (completed years)
13. RESIDENCE (House No., St., Barangay) (City/Municipality) (Province) (Country)

FATHER

14. NAME (First) (Middle) (Last)
15. CITIZENSHIP
16. RELIGION/RELIGIOUS SECT
17. OCCUPATION
18. AGE at the time of this birth (completed years)

[source]

日 本 国 JAPAN

旅 券 PASSPORT

型/Type P
発行国/Issuing country JPN
旅券番号/Passport No. XS1234567

SPECIMEN

姓/Surname GAIMU
名/Given name SAKURA
国籍/Nationality JAPAN
生年月日/Date of birth 20 FEB 1979
性別/Sex F
本籍/Registered Domicile KANAGAWA
発行年月日/Date of issue 01 JAN 2006
有効期間満了日/Date of expiry 01 JAN 2011
発行官庁/Authority MINISTRY OF FOREIGN AFFAIRS
所持人自署/Signature of bearer SPECIMEN
外務さくら

P<JPNGAIMU<<SAKURA<<<<<<<<<<<<<<<<<<<<<<<<
XS12345673JPN7902206F1101018<<<<<<<<<<<<<<00

[source]

MOTIVATION

We need a programming language that having:

- Good memory efficiency and performance
- Shared resources (global counters, caches, configuration, etc)
- Preloading data that helps initialize configurations before object creation.

Idea: We need statics variables that are shared among all instances!

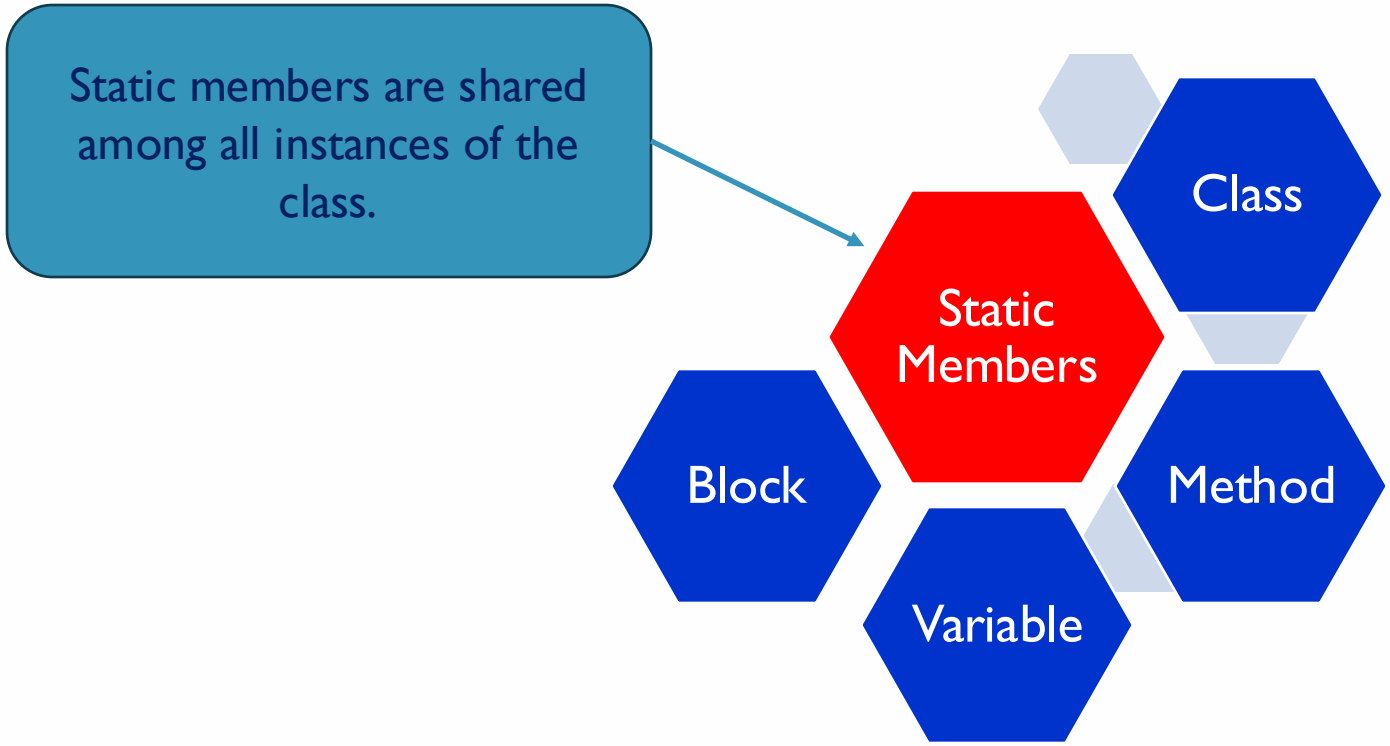
LEARNING OUTCOMES

Upon the completion of this lecture, students will be able to:

- understand static keyword used for class, variable, method and block.
- understand, use, and create static blocks/static variables/ static methods.
- understand, use, and create static class, including statics nested class.

OVERVIEW OF THE STATIC MEMBERS

- In Java, static members refer to fields (variables) and methods that belong to a class rather than to any specific instance of that class.
- Can be accessed without an object.



STATIC BLOCK

STATIC BLOCK: EXAMPLE 01

Static blocks are "a block of codes" with a `static` keyword. In general, these are used to initialize **the static members**. JVM executes static blocks **before** the main method at the time of class loading.

```
1 public class MyClass {
2     static{
3         System.out.println("Hello this is a static block");
4     }
5     public static void main(String args[]){
6         System.out.println("This is main method");
7     }
8 }
```

```
Hello this is a static block
This is main method
```

JVM executes static blocks **before** the main method at the time of class loading.

STATIC BLOCK: EXAMPLE 01

- Executes before the main method.
- Multiple static blocks
- No need for an object
- Static blocks execute in the order they appear

STATIC BLOCK: EXAMPLE 02

```
public class MyStaticBlock {

    int numbOfStudents = 50;
    String major = "CHS";

    //First Static block
    static{
        System.out.println("CECS");
    }

    //Second static block
    static{
        System.out.println("CBM");
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        MyStaticBlock obj = new MyStaticBlock();

        System.out.println("Number of Students: " + obj.numbOfStudents);
        System.out.println("College: " + obj.major);
    }
}
```

Ouput?

STATIC BLOCK: EXAMPLE 02

```
public class MyStaticBlock {

    int numbOfStudents = 50;
    String major = "CHS";

    //First Static block
    static{
        System.out.println("CECS");
    }

    //Second static block
    static{
        System.out.println("CBM");
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        MyStaticBlock obj = new MyStaticBlock();

        System.out.println("Number of Students: " + obj.numbOfStudents);
        System.out.println("College: " + obj.major);
    }
}
```

Ouput?

```
CECS
CBM
Number of Students: 50
College: CHS
```

STATIC VARIABLES

INSTANCE VARIABLE (REVISIT)

- Instance variables in Java are non-static variables which are defined in a class outside any method, constructor or a block.
- Each instantiated object of the class has a separate copy or instance of that variable.
- They are used to store unique data for each instance of the class.

INSTANCE VARIABLE EXAMPLE

What will be output?

```
public class MyStaticVariable {

    int salary;  // Instance variable (not static)

    public static void main(String[] args) {
        MyStaticVariable staff1 = new MyStaticVariable();
        MyStaticVariable staff2 = new MyStaticVariable();

        staff1.salary = 1000;
        staff2.salary = 2000;

        System.out.println(staff1.salary);
        System.out.println(staff2.salary);
    }
}
```

INSTANCE VARIABLE EXAMPLE

What will be output?

```
public class MyStaticVariable {

    int salary;  // Instance variable (not static)

    public static void main(String[] args) {
        MyStaticVariable staff1 = new MyStaticVariable();
        MyStaticVariable staff2 = new MyStaticVariable();

        staff1.salary = 1000;
        staff2.salary = 2000;

        System.out.println(staff1.salary);
        System.out.println(staff2.salary);
    }
}
```

Output:

1000
2000

The variable `salary` is an instance variable (not static), meaning each object has its own copy.

STATIC VARIABLE EXAMPLE 1

```
public class MyStaticVariable {

    static int salary;

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        MyStaticVariable staff1 = new MyStaticVariable();
        MyStaticVariable staff2 = new MyStaticVariable();

        staff1.salary = 1000;
        staff2.salary = 2000;

        System.out.println(staff1.salary);
        System.out.println(staff2.salary);
    }
}
```

What happen if `salary` were declared as `static`?

Output?

STATIC VARIABLE EXAMPLE 1

```
public class MyStaticVariable {

    static int salary;

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        MyStaticVariable staff1 = new MyStaticVariable();
        MyStaticVariable staff2 = new MyStaticVariable();

        staff1.salary = 1000;
        staff2.salary = 2000;

        System.out.println(staff1.salary);
        System.out.println(staff2.salary);
    }
}
```

What happen if `salary` were declared as `static`?

Output?

```
2000
2000
```

STATIC VARIABLE

- A static variable is a class-level variable that is shared among all instances of the class. It belongs to the class itself, rather than to any individual instance of the class.
- To define a static variable, use the **`static`** keyword before the data type.

```
class MyClass {
    static int myStaticVar = 10;  // Static variable
}
```

STATIC VARIABLE EXAMPLE 2

```
public class MyStaticVariable {

    static int salary;

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        MyStaticVariable staff1 = new MyStaticVariable();
        MyStaticVariable staff2 = new MyStaticVariable();

        MyStaticVariable.salary = 2000;
        //staff1.salary = 2000;
        //staff2.salary = 2000;

        System.out.println(staff1.salary);
        System.out.println(staff2.salary);
    }
}
```

Output?

STATIC VARIABLE EXAMPLE 2

```
public class MyStaticVariable {

    static int salary;

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        MyStaticVariable staff1 = new MyStaticVariable();
        MyStaticVariable staff2 = new MyStaticVariable();

        MyStaticVariable.salary = 2000;
        //staff1.salary = 2000;
        //staff2.salary = 2000;

        System.out.println(staff1.salary);
        System.out.println(staff2.salary);
    }
}
```

Output?

2000
2000

STATIC VARIABLE EXAMPLE 3: PRACTICE

```
public class MyStaticBlock {

    static int numbOfStudents = 60;
    static String major = "CHS";

    static{
        System.out.println("CECS");
        numbOfStudents = 71;
        major = "Computer Science";
    }

    static{
        System.out.println("CBM");
        numbOfStudents = 82;
        major = "Hospitality";
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        System.out.println("Number of Students: " + numbOfStudents);
        System.out.println("College: " + major);
    }
}
```

Rewrite the program and execute it?

STATIC VARIABLE EXAMPLE 3: PRATICE

```
public class MyStaticBlock {

    static int numbOfStudents = 60;
    static String major = "CHS";

    static{
        System.out.println("CECS");
        numbOfStudents = 71;
        major = "Computer Science";
    }

    static{
        System.out.println("CBM");
        numbOfStudents = 82;
        major = "Hospitality";
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        System.out.println("Number of Students: " + numbOfStudents);
        System.out.println("College: " + major);
    }
}
```

```
CECS
CBM
Number of Students: 82
College: Hospitality
```

STATIC METHODS

INSTANCE METHOD *VS* STATIC METHOD

Instance Methods:

- Require an object of its class to be created before calling it.

Static Methods:

- Belong to a class rather than an instance of a class.
- Expected to work without an instance.

CHARACTERISTICS OF STATIC METHODS

1. They are declared using the "`static`" keyword.
2. They are shared among all instances of the class.
3. They can be called using the class name, without the need for an object reference.
4. They can access only static variables or other static methods of the same class.
5. They cannot access instance variables or instance methods of the same class directly.
6. They can be overridden in a subclass, but the method in the superclass is still executed if the method is called using the superclass reference.

STATIC METHOD EXAMPLE

```java
class Student {
    int studentID;
    String name;
    static String college = "CBM";

    // constructor to initialize the variable
    Student(int studentID, String name) {
        this.studentID = studentID;
        this.name = name;
    }

    // static method
    static void changeCollege() {
        college = "CECS";
    }

    // instance method to display values
    void displayInfo() {
        System.out.println(studentID + ", " + name + ", " + college);
    }
}
```

```java
public class MyStaticClassMember {
    public static void main(String args[]) {

        Student.changeCollege();

        Student s1 = new Student(1955434, "Ali Baba");
        s1.displayInfo();
    }
}
```

calling static method

We can call this method without any object: when a member is static it becomes class level.

```
1955434, Ali Baba, CECS
```

STATIC NESTED CLASS

WHY USE NESTED CLASSES?

- It is a way of logically grouping classes that are only used in one place.
- It increases encapsulation.
- It can lead to more readable and maintainable code.

Further Reading: https://docs.oracle.com/javase/tutorial/java/javaOO/nested.html

NESTED CLASSES

- A class within another class.
- Two types: non-static and static

```
class OuterClass {
    …
    class InnerClass {
        …
    }
}
```

```
class OuterClass {
    …
    static class StaticNestedClass {
        …
    }
}
```

NON-STATIC NESTED CLASS

- Non-static nested classes (inner classes) have access to other members of the enclosing class (even if they are declared private).

```
class OuterClass {
    …
    class InnerClass {
        …
    }
}
```

Inner class cannot define any static members itself. Why?

Because an inner class is associated with an instance!

NON-STATIC NESTED CLASS EXAMPLE 1

```
public class MyOuterClass {

    int distance = 200;

    class MyInnerClass{
        void printInfo() {
            System.out.println(distance+"KM");
        }
    }

    public static void main(String[] args) {

        MyOuterClass objOuter = new MyOuterClass();
        MyOuterClass.MyInnerClass objInner = objOuter.new MyInnerClass();

        objInner.printInfo();
    }
}
```

non-static nested class

To instantiate an inner class, you must first instantiate the outer class. Then, create the inner object within the outer object.

```
200KM
```

Non-static Nested Class Example 2

```
class VinUnian{
    int professorCount = 200;
    String name = "VinUniversity";

    class Staff{
        int staffCount = 350;
    }

    class Student{
        int studentCount = 600;
        String majors[] = {"CHS", "CBM", "CECS"};
    }
}
public class NestedClassDemo {

    public static void main(String[] args) {
        VinUnian v = new VinUnian();
        VinUnian.Staff s = v.new Staff();
        VinUnian.Student k = v.new Student();

        int count = v.professorCount+s.staffCount+k.studentCount;
        System.out.println("There are " + count +" Vinunians in "+v.name);

        for (String major : k.majors) {
            System.out.println(major);
        }
    }
}
```

Ouput?

Non-static Nested Class Example 2

```
class VinUnian{
    int professorCount = 200;
    String name = "VinUniversity";

    class Staff{
        int staffCount = 350;
    }

    class Student{
        int studentCount = 600;
        String majors[] = {"CHS", "CBM", "CECS"};
    }
}
public class NestedClassDemo {

    public static void main(String[] args) {
        VinUnian v = new VinUnian();
        VinUnian.Staff s = v.new Staff();
        VinUnian.Student k = v.new Student();

        int count = v.professorCount+s.staffCount+k.studentCount;
        System.out.println("There are " + count +" Vinunians in "+v.name);

        for (String major : k.majors) {
            System.out.println(major);
        }
    }
}
```

```
There are 1150 Vinunians in VinUniversity
CHS
CBM
CECS
```

STATIC NESTED CLASS

- Static nested classes cannot access to other members of the enclosing class.
- As a member of the Outerclass, a nested class can be declared private, public, protected, package private.
- Note that Outerclass can only be declared public or package private.

```
class OuterClass {
    …
    static class StaticNestedClass {
        …
    }
}
```

STATIC NESTED CLASS EXAMPLE

```
public class MyOuterClass {

    private static String myUni = "VinUniversity";
    static int year = 2020;

    static class MyStaticNestedClass{

        private static String location = "Hanoi";

        public void dispInfo() {
            System.out.println(myUni);
            System.out.println(year);
        }
    }

    public static void main(String[] args) {
        MyStaticNestedClass obj = new MyStaticNestedClass();
        obj.dispInfo();

        System.out.println(MyStaticNestedClass.location);
    }
}
```

static nested class (associated with its outer class)

```
VinUniversity
2020
Hanoi
```

TAKE-HOME MESSAGE

✓ Use static variables when data should be shared across all instances.

✓ Use static methods when behavior is independent of any instance.

✓ Use static nested classes when they do not need access to outer class instance members.

✗ Avoid static when each instance needs its own copy of data.

THANK YOU!

The next lecture will be delivered by

Professor Sungdeok Cha